

E-commerce Customer Analytics

Data analytics project focused on understanding customer behavior, identifying valuable customers, predicting churn, and uncovering the products that drive revenue for an online retail business.

Business Scenario

An online retailer has thousands of customers but doesn't know:

- Who the **valuable customers** are
- Who is **about to stop buying** (churn risk)
- Which **products drive revenue**

As a Data Analyst, the goal of this project is to **improve customer retention** and **increase revenue** through data-driven insights.

Dataset

The project uses the following relational datasets:

Table	Description
customers	Customer profile and demographic details
orders	Order-level transaction data
order_items	Line-item details for each order
products	Product catalog and category information
payments	Payment method and transaction status
reviews	Customer product reviews and ratings

Update this section with actual source/link of the dataset (e.g., Kaggle, company database, etc.)

? Business Questions

Revenue

- Monthly sales trend
- Year-over-year (YoY) growth
- Average Order Value (AOV)

Customers

- Repeat purchase rate
- Customer Lifetime Value (CLV)

- New vs Returning customers
- Top 10 customers by revenue

Products

- Best-selling categories
- Low-performing products
- Most profitable products

Customer Segmentation — RFM Analysis

Customers are segmented using **RFM Analysis**:

Metric	Meaning
Recency (R)	How recently did the customer purchase?
Frequency (F)	How often do they purchase?
Monetary (M)	How much have they spent?

Customer Segments

-  **Champions**
-  **Loyal Customers**
-  **Potential Loyalists**
-  **At Risk**
-  **Lost Customers**

Python Analysis

The following analyses are performed using Python:

- **RFM Scoring** – Quantile-based scoring of Recency, Frequency, and Monetary values
- **K-Means Clustering** – Unsupervised clustering to validate/refine RFM segments
- **Cohort Analysis** – Tracking customer retention over time by acquisition month
- **Customer Retention Analysis** – Measuring repeat purchase and churn behavior

Tech Stack

- `pandas`, `numpy` – Data wrangling
- `matplotlib`, `seaborn` – Visualization
- `scikit-learn` – K-Means clustering
- `jupyter notebook` – Analysis environment

Dashboard

Interactive dashboard(s) covering:

1. **Revenue Dashboard** – Sales trends, growth, AOV
2. **Customer Segments Dashboard** – RFM segment distribution and value
3. **Product Performance Dashboard** – Category and product-level insights

4. Retention Dashboard – Cohort retention curves and churn trends

Add your dashboard tool here (Power BI / Tableau / Looker Studio) and a screenshot or live link once built.

💡 Business Recommendation

A small percentage of customers generate a large share of revenue (Pareto principle).

Personalized loyalty programs targeting these high-value (“Champion” and “Loyal”) customers could significantly improve retention and long-term revenue.

📁 Project Structure

```
ecommerce-customer-analytics/  
├── data/                # Raw and processed datasets  
├── notebooks/           # Jupyter notebooks (RFM, clustering, cohort analysis)  
├── dashboard/           # Dashboard files/screenshots  
├── src/                 # Python scripts (data cleaning, RFM, clustering)  
├── outputs/             # Reports, visualizations, exported results  
├── requirements.txt      # Python dependencies  
└── README.md            # Project documentation
```

⚙️ Setup & Installation

1. Clone the repository

```
git clone https://github.com/<your-username>/ecommerce-customer-analytics.git  
cd ecommerce-customer-analytics
```

2. Create a virtual environment

```
python -m venv venv  
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Run the analysis

```
jupyter notebook notebooks/rfm_analysis.ipynb
```

🔗 Usage

- Place raw datasets inside the `data/` folder
- Run notebooks in `notebooks/` in order:
 1. Data cleaning & preprocessing
 2. RFM scoring
 3. K-Means clustering

4. Cohort & retention analysis

- Generated outputs (charts, tables) will be saved in `outputs/`
- Load dashboard files from `dashboard/` into your BI tool of choice

Key Insights (Sample)

Update this section once analysis is complete

- X% of customers are classified as “Champions” and contribute Y% of total revenue
- Average customer retention rate is Z% month-over-month
- Top category “___” drives the highest revenue share

Contributing

Contributions are welcome! Please open an issue or submit a pull request for any improvements.

License

This project is licensed under the MIT License.

Contact

Author: Your Name **Email:** your.email@example.com **LinkedIn:** [your-linkedin](#)

Appendix: Full Python Source Code (rfm_analysis.py)

```
"""
E-commerce Customer Analytics
=====
Performs:
  1. Data loading & merging (customers, orders, order_items, products, payments, reviews)
  2. RFM Scoring (Recency, Frequency, Monetary)
  3. Customer Segmentation (Champions, Loyal, Potential Loyalists, At Risk, Lost)
  4. K-Means Clustering (validate/refine RFM segments)
  5. Cohort Analysis (monthly retention by acquisition cohort)
  6. Customer Retention Analysis (repeat purchase rate, churn)

Expected input CSVs (place inside ./data/):
- customers.csv      : customer_id, signup_date, ...
- orders.csv         : order_id, customer_id, order_date, order_status, ...
- order_items.csv    : order_id, product_id, quantity, price
- products.csv       : product_id, category, ...
- payments.csv       : order_id, payment_value, payment_type
- reviews.csv        : order_id, review_score

Output:
- outputs/rfm_scores.csv
- outputs/rfm_segments.csv
- outputs/kmeans_clusters.csv
- outputs/cohort_retention.csv
- outputs/summary_report.txt
"""
```

```

import os
import pandas as pd
import numpy as np
from datetime import datetime
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

DATA_DIR = "data"
OUTPUT_DIR = "outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# -----
# 1. DATA LOADING
# -----
def load_data():
    """Load and merge raw datasets into a single transaction-level dataframe."""
    customers = pd.read_csv(f"{DATA_DIR}/customers.csv", parse_dates=["signup_date"])
    orders = pd.read_csv(f"{DATA_DIR}/orders.csv", parse_dates=["order_date"])
    order_items = pd.read_csv(f"{DATA_DIR}/order_items.csv")
    products = pd.read_csv(f"{DATA_DIR}/products.csv")
    payments = pd.read_csv(f"{DATA_DIR}/payments.csv")

    # Merge order_items with orders, products, payments
    df = order_items.merge(orders, on="order_id", how="left")
    df = df.merge(products, on="product_id", how="left")
    df = df.merge(
        payments.groupby("order_id")["payment_value"].sum().reset_index(),
        on="order_id", how="left"
    )
    df = df.merge(customers, on="customer_id", how="left")

    # Line total (fallback to quantity * price if payment_value missing)
    df["line_total"] = df["quantity"] * df["price"]

    return df, customers, orders

# -----
# 2. RFM SCORING
# -----
def compute_rfm(df, snapshot_date=None):
    """Compute Recency, Frequency, Monetary values per customer and score them 1-5."""
    if snapshot_date is None:
        snapshot_date = df["order_date"].max() + pd.Timedelta(days=1)

    rfm = df.groupby("customer_id").agg(
        recency=("order_date", lambda x: (snapshot_date - x.max()).days),
        frequency=("order_id", "nunique"),
        monetary=("line_total", "sum")
    ).reset_index()

    # Score 1-5 using quantiles (5 = best)
    rfm["R_score"] = pd.qcut(rfm["recency"], 5, labels=[5, 4, 3, 2, 1]).astype(int)
    rfm["F_score"] = pd.qcut(rfm["frequency"].rank(method="first"), 5, labels=[1, 2, 3, 4, 5]).astype(int)
    rfm["M_score"] = pd.qcut(rfm["monetary"].rank(method="first"), 5, labels=[1, 2, 3, 4, 5]).astype(int)

    rfm["RFM_score"] = rfm["R_score"].astype(str) + rfm["F_score"].astype(str) + rfm["M_score"].astype(str)
    rfm["RFM_total"] = rfm[["R_score", "F_score", "M_score"]].sum(axis=1)

    return rfm

def segment_customers(rfm):
    """Classify customers into segments based on RFM scores."""
    def classify(row):
        r, f, m = row["R_score"], row["F_score"], row["M_score"]
        if r >= 4 and f >= 4 and m >= 4:
            return "Champions"
        elif r >= 3 and f >= 3:
            return "Loyal Customers"
        elif r >= 4 and f <= 2:
            return "Potential Loyalists"
        elif r <= 2 and f >= 3:
            return "At Risk"
        elif r <= 2 and f <= 2:
            return "Lost Customers"
        else:
            return "Others"

    rfm["segment"] = rfm.apply(classify, axis=1)
    return rfm

```

```

# -----
# 3. K-MEANS CLUSTERING
# -----
def kmeans_clustering(rfm, n_clusters=5):
    """Cluster customers using K-Means on scaled RFM values."""
    features = rfm[["recency", "frequency", "monetary"]].copy()

    scaler = StandardScaler()
    scaled = scaler.fit_transform(features)

    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
    rfm["cluster"] = kmeans.fit_predict(scaled)

    return rfm, kmeans

# -----
# 4. COHORT ANALYSIS
# -----
def cohort_analysis(orders):
    """Build a monthly cohort retention table based on customer's first order month."""
    orders = orders.copy()
    orders["order_month"] = orders["order_date"].dt.to_period("M")

    first_purchase = orders.groupby("customer_id")["order_month"].min().reset_index()
    first_purchase.columns = ["customer_id", "cohort_month"]

    orders = orders.merge(first_purchase, on="customer_id", how="left")
    orders["cohort_index"] = (
        (orders["order_month"].dt.year - orders["cohort_month"].dt.year) * 12
        + (orders["order_month"].dt.month - orders["cohort_month"].dt.month)
    )

    cohort_data = orders.groupby(["cohort_month", "cohort_index"])["customer_id"] \
        .nunique().reset_index()

    cohort_pivot = cohort_data.pivot(index="cohort_month", columns="cohort_index", values="customer_id")
    cohort_size = cohort_pivot.iloc[:, 0]
    retention = cohort_pivot.divide(cohort_size, axis=0).round(3) * 100

    return retention

# -----
# 5. RETENTION METRICS
# -----
def retention_metrics(orders):
    """Compute repeat purchase rate and new vs returning customer counts."""
    order_counts = orders.groupby("customer_id")["order_id"].nunique()
    repeat_customers = (order_counts > 1).sum()
    total_customers = order_counts.shape[0]
    repeat_purchase_rate = round(repeat_customers / total_customers * 100, 2)

    return {
        "total_customers": total_customers,
        "repeat_customers": repeat_customers,
        "repeat_purchase_rate%": repeat_purchase_rate
    }

# -----
# MAIN PIPELINE
# -----
def main():
    print("Loading data...")
    df, customers, orders = load_data()

    print("Computing RFM scores...")
    rfm = compute_rfm(df)
    rfm = segment_customers(rfm)
    rfm.to_csv(f"{OUTPUT_DIR}/rfm_segments.csv", index=False)

    print("Running K-Means clustering...")
    rfm, model = kmeans_clustering(rfm)
    rfm.to_csv(f"{OUTPUT_DIR}/kmeans_clusters.csv", index=False)

    print("Building cohort retention table...")
    retention_table = cohort_analysis(orders)
    retention_table.to_csv(f"{OUTPUT_DIR}/cohort_retention.csv")

```

```

print("Calculating retention metrics...")
metrics = retention_metrics(orders)

# Summary report
with open(f"{OUTPUT_DIR}/summary_report.txt", "w") as f:
    f.write("E-COMMERCE CUSTOMER ANALYTICS - SUMMARY\n")
    f.write("=" * 45 + "\n\n")
    f.write(f"Total Customers: {metrics['total_customers']}\n")
    f.write(f"Repeat Customers: {metrics['repeat_customers']}\n")
    f.write(f"Repeat Purchase Rate: {metrics['repeat_purchase_rate_%']}%\n\n")
    f.write("Segment Distribution:\n")
    f.write(rfm["segment"].value_counts().to_string())
    f.write("\n\nTop 10 Customers by Monetary Value:\n")
    f.write(rfm.sort_values("monetary", ascending=False).head(10)[
        ["customer_id", "recency", "frequency", "monetary", "segment"]
    ].to_string(index=False))

print(f"Done. Outputs saved in ./{OUTPUT_DIR}/")

if __name__ == "__main__":
    main()

```